

Punto 3.4 — Proyecciones a DTOs

Contexto del proyecto

Este checkpoint continúa `M03/PROYECTO/3.3`. Se mantiene la separación `Domain/Application/Infrastructure/Console` y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Proyecciones a DTOs** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.4
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerOrdenesConPlanchas`, `ObtenerOrdenesConDetalle`, `ObtenerOrdenesCompletas`, `ObtenerSqlProyeccionNavegacion`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ProyeccionesDtoUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ProyeccionesDtoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES A DTOs ===");
        var conPlanchas = _unidad.Ordenes.ObtenerOrdenesConPlanchas();
        var conDetalle = _unidad.Ordenes.ObtenerOrdenesConDetalle();
        var completas = _unidad.Ordenes.ObtenerOrdenesCompletas();
        var primera = completas.Single(o => o.NumeroOrden == "OF-2024-0001");
        if (conPlanchas.Count != 5 || conDetalle.Count != 5 || completas.Count != 5 ||
            primera.Planchas.Count != 2 || primera.Detalle is null) throw new
            InvalidOperationException("Proyecciones DTO inesperadas.");
        Console.WriteLine($"OF-2024-0001 -> planchas: {primera.Planchas.Count}, detalle:
            {primera.Detalle.ComposicionQuimica}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccionNavegacion());
    }
}

```

Explicación línea a línea del caso de uso 3.4

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class ProyeccionesDtoUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== PROYECCIONES A DTOs ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var conPlanchas = _unidad.Ordenes.ObtenerOrdenesConPlanchas();` → Proyecta las órdenes a DTOs que contienen una colección anidada de planchas.

Línea 14: `var conDetalle = _unidad.Ordenes.ObtenerOrdenesConDetalle();` → Proyecta cada orden junto con su DTO de detalle, contemplando que la navegación de referencia pueda ser nula.

Línea 15: `var completas = _unidad.Ordenes.ObtenerOrdenesCompletas();` → Combina en un único DTO la información principal, la colección de planchas y el detalle de cada orden.

Línea 16: `var primera = completas.Single(o => o.NumeroOrden == "OF-2024-0001");` → Selecciona de forma inequívoca la orden OF-2024-0001 del resultado proyectado para comprobar sus datos anidados.

Línea 17: `if (conPlanchas.Count != 5 || conDetalle.Count != 5 || completas.Count != 5 || primera.Planchas.Count != 2 || primera.Detalle is null) throw new`

`InvalidOperationException("Proyecciones DTO inesperadas.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 18: `Console.WriteLine($"0F-2024-0001 -> planchas: {primera.Planchas.Count}, detalle: {primera.Detalle.ComposicionQuimica}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 19: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccionNavegacion());` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

| Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ProyeccionesDtoUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesDtoUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.4 OK");
```

Explicación línea a línea de Program.cs 3.4

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa `AceriaDbContext` para preparar la base de demostración desde el `composition root`.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por `Migrate`, `ChangeTracker` y las operaciones de consulta del `checkpoint`.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa `ConfigurationBuilder` y las extensiones necesarias para cargar `appsettings.json` y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa `ServiceCollection`, `AddScoped`, `CreateScope` y `GetRequiredService` para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar `appsettings.json`.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<ProyeccionesDtoUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesDtoUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.4 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.4 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: No comprobar una relación opcional antes de proyectarla rompe la semántica de null del modelo.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/ProyeccionesDtoUseCase.cs`

Marcador: `ERROR CONTROLADO M03 3.4 - RELACION OPCIONAL SIN COMPROBAR NULL`

```
// Demuestra el Paso 9 usando una orden cuyo Detalle opcional no existe.
var sinDetalleError = _unidad.Ordenes.ObtenerOrdenesCompletras()
    .Single(o => o.NumeroOrden == "0F-2024-0005");

Console.WriteLine(sinDetalleError.Detalle!.ComposicionQuimica);
```

Comprobación esperada: la relación opcional sin comprobación produce `NullReferenceException` de forma controlada.

Laboratorio adicional del punto 3.4

Diagnóstico técnico

Las proyecciones a DTO deben decidir qué ocurre con relaciones opcionales. `Detalle` puede ser `null`; el DTO también debe admitirlo. Las colecciones relacionadas se convierten a DTOs de lectura, no se filtran entidades de Infrastructure hacia Application.

Reto resuelto y verificación adicional

Selecciona `0F-2024-0001` en `OrdenCompletaDto`: debe tener dos planchas y detalle. Comprueba también una relación opcional ausente sin provocar `NullReferenceException`.

Errores comunes revisados

Error	Causa	Solución
<code>NullReferenceException</code> en DTO anidado	No se comprobó si la entidad relacionada es <code>null</code>	Usar operador ternario antes de proyectar
Tipo de colección incompatible	El DTO declara <code>List<T></code> pero la proyección no produce una lista compatible	Materializar con <code>ToList()</code> cuando lo exija el tipo de destino
Propiedad no proyectada	Se accede a una propiedad que no está en la proyección	Proyectar todas las propiedades necesarias
<code>GroupBy</code> tratado siempre como <code>GROUP BY SQL</code>	Se presupone que cualquier forma de agrupación tiene la misma traducción	Para el patrón relacional usar clave + agregados; para otros <i>shapes</i> , inspeccionar la traducción real
<code>SelectMany</code> con colección vacía	La entidad principal no aparece en el resultado	Usar <code>DefaultIfEmpty</code>
Average sobre colección vacía	La semántica depende de si el tipo es anulable	Proyectar a nullable y definir el resultado, o validar que existan elementos

Analogía operativa

Las proyecciones a DTOs en una acería son como los formularios estandarizados que se usan para transferir información entre departamentos. En lugar de enviar la carpeta completa de una orden con todos sus documentos, se envía un formulario con los campos que el departamento receptor necesita. El formulario tiene un nombre, un formato fijo y se puede archivar. Proyectar a un DTO con inicializador de objeto es como rellenar un formulario campo a campo. Proyectar a un DTO con constructor es como usar un formulario preimpreso que garantiza que todos los campos estén rellenos. Proyectar una colección de navegación dentro de un DTO es como adjuntar una lista de planchas al formulario de la orden. Proyectar un DTO anidado es como incluir un formulario de detalle dentro del formulario principal. `SelectMany` es como aplanar varias listas en una sola. `GroupBy` con agregaciones es como pedir un resumen por cliente con totales. Así funcionan las proyecciones a DTOs en EF Core: se seleccionan solo los datos necesarios, se estructuran en formularios estandarizados y se transfieren entre capas de forma eficiente.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Crea un DTO que incluya una lista de planchas y un detalle opcional sin exponer entidades.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: src/AceriaData.Application/ProyeccionesDtoUseCase.cs

Marcador: RETO M03 3.4 - DTO CON PLANCHAS Y DETALLE OPCIONAL

```
// Demuestra el Paso 10 y el laboratorio adicional sin exponer entidades.
var completasReto = _unidad Ordenes.ObtenerOrdenesCompletas();
var conDetalleReto = completasReto.Single(o => o.NumeroOrden == "OF-2024-0001");
var sinDetalleReto = completasReto.Single(o => o.NumeroOrden == "OF-2024-0005");

if (conDetalleReto.Planchas.Count != 2 || conDetalleReto.Detalle is null)
    throw new InvalidOperationException("Reto 3.4: OF-2024-0001 debe conservar dos planchas y detalle.");

if (sinDetalleReto.Detalle is not null)
    throw new InvalidOperationException("Reto 3.4: OF-2024-0005 debe conservar Detalle == null.");

Console.WriteLine($"Reto 3.4 OK | OF-2024-0001 planchas: {conDetalleReto.Planchas.Count} | OF-2024-0005 detalle: null");
```

Comprobación esperada: DTO con colección de planchas y detalle opcional sin exponer entidades.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.4 OK, el siguiente estado parte exactamente de esta solución y añade

Consultas de agregación: Count, Sum, Average, Min y Max.
